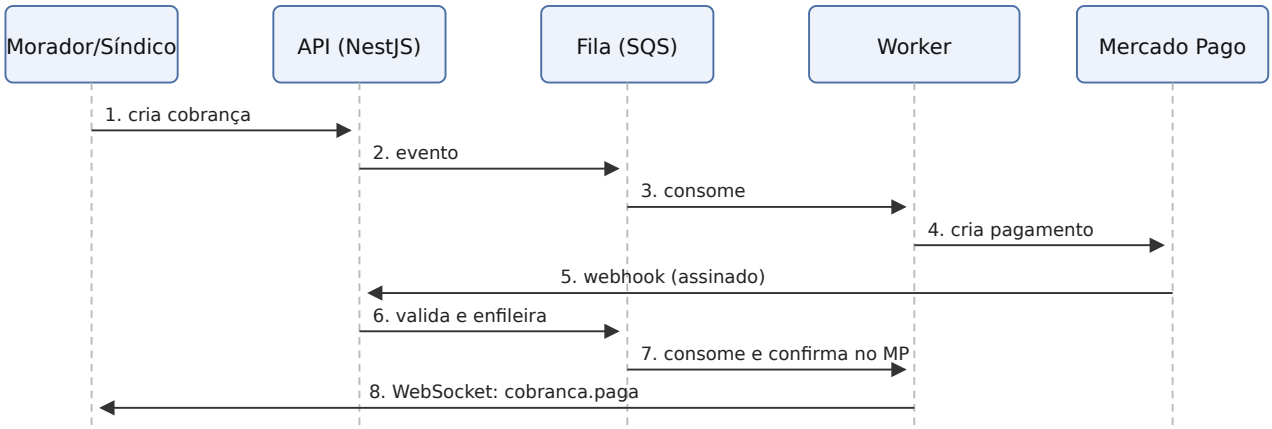
Zeluvi – Mapeamento de Integrações

Complementa o Documento de System Design e a Definição Arquitetural. Cobre as duas integrações externas do sistema: a API de pagamento (Mercado Pago) e a comunicação em tempo real (WebSocket). O backend segue a Definição Arquitetural: NestJS com TypeScript, em réplicas stateless.

1. Visão geral

Integração	Sentido	Para que serve	Módulo
Mercado Pago (Pix e boleto)	Saída (cria cobrança) e entrada (webhook de status)	Cobrança de taxas e situação de pagamento do morador (RF09 da Fase 2)	Financeiro, via worker
WebSocket (Socket.IO no NestJS)	Tempo real, nos dois sentidos	Avisos e atualizações ao vivo no web	Comunicação, Ocorrências, Serviços e Financeiro

O mobile recebe os mesmos avisos por notificação push, como definido na Definição Arquitetural. Esse canal não é detalhado aqui.



Fluxo de cobrança e confirmação de pagamento (resumo)

2. Mercado Pago

2.1 Princípios

- **Não guardamos dados de cartão.** O checkout é do Mercado Pago. O sistema guarda apenas o identificador do pagamento, valor, vencimento, status e data de pagamento.
- **Não confiamos só no webhook.** Ao receber a notificação, o sistema valida a assinatura e consulta o pagamento na API do Mercado Pago para confirmar o status.
- **Segredos fora do código.** O segredo do webhook e o access token ficam no AWS Secrets Manager.

- **Dados do pagador (LGPD).** Para gerar Pix ou boleto, o Mercado Pago pode exigir dados do pagador, como nome e CPF. Enviamos só o necessário, registramos o consentimento do morador e não gravamos o CPF em logs.

2.2 Fluxo de cobrança

1. O síndico (ou uma rotina mensal) cria a cobrança para a unidade. A API grava com status `PENDENTE` e publica um evento na fila.
2. Um worker chama a API do Mercado Pago para criar o pagamento (Pix ou boleto), com chave de idempotência derivada do id interno da cobrança. Uma nova tentativa não gera pagamento duplicado.
3. O sistema salva o id do pagamento e mostra o código Pix ou o boleto ao morador.
4. Quando o morador paga, o Mercado Pago envia um webhook para `POST /v1/webhooks/mercadopago`.
5. A API valida a assinatura, responde `200` rapidamente e publica o evento na fila. Não processa nada dentro da requisição.
6. Um worker consulta o pagamento no Mercado Pago, confirma o status e atualiza a cobrança para `PAGA`.
7. O evento `cobranca.paga` é emitido pelo WebSocket ao morador e ao síndico, e o registro vai para a trilha de auditoria.

2.3 Validação do webhook

- O header `x-signature` tem o formato `ts={timestamp},v1={hash}`.
- O hash é um HMAC-SHA256 em hexadecimal, calculado com o segredo da aplicação sobre um texto que combina o `data.id` da notificação, o header `x-request-id` e o `ts`. A comparação é feita em tempo constante.
- Assinatura inválida: resposta `401` e registro para monitoramento.
- O Mercado Pago pode reenviar a mesma notificação. O `id` da notificação é gravado numa tabela de notificações processadas, e um reenvio é ignorado.

2.4 Conciliação

Um webhook pode se perder. Por isso uma rotina agendada consulta no Mercado Pago os pagamentos que continuam `PENDENTE` além de um prazo definido e corrige o status. Assim, nenhuma cobrança paga fica como pendente por falha de entrega.

2.5 Falhas

Situação	Resposta
Mercado Pago fora do ar ao criar a cobrança	A cobrança fica na fila e o worker tenta de novo com espera crescente. O restante do sistema continua funcionando.
Timeout após enviar a criação	Nova tentativa com a mesma chave de idempotência, sem pagamento duplicado.
Webhook duplicado	Ignorado pela tabela de notificações processadas.
Webhook não entregue	Corrigido pela rotina de conciliação.
Assinatura inválida	Rejeitado com `401` e alerta.

2.6 Volume

Pagamentos são eventos mensais por unidade e ficam muito abaixo da meta de 1 milhão de escritas. Não pesam no dimensionamento do banco. O que importa aqui é a confiabilidade: não cobrar duas vezes e não perder uma confirmação.

3. WebSocket

3.1 Tecnologia

- **Gateway no NestJS** (`@nestjs/websockets` com Socket.IO), na mesma base de código da API.
- **Adaptador Redis** (`@socket.io/redis-adapter`) sobre o ElastiCache. Cada conexão fica presa a uma réplica, e o adaptador leva o evento a todas elas, para que o cliente receba a mensagem em qualquer réplica.

3.2 Autenticação e salas

- O cliente envia o token JWT no handshake. Sem token válido, a conexão é recusada.
- Ao conectar, o usuário entra nas salas `usuario:{id}` e `condominio:{id}`. O síndico entra também em `condominio:{id}:admin`.
- O id do condomínio vem do token, nunca do cliente. Um usuário não entra na sala de outro condomínio.

3.3 Eventos

Evento	Destino	Quando
`aviso.publicado`	`condominio:{id}`	Síndico publica comunicado
`ocorrencia.status_alterado`	`usuario:{id}`	Síndico muda o status da ocorrência
`reserva.atualizada`	`usuario:{id}`	Mudança de estado da reserva
`encomenda.recebida`	`usuario:{id}`	Portaria registra encomenda
`cobranca.paga`	`usuario:{id}` e `condominio:{id}:admin`	Pagamento confirmado

3.4 Caminho de um evento

1. Uma ação gera um evento de domínio na fila (SQS).
2. Um worker consome o evento e pede ao gateway que emita.
3. O gateway publica no Redis pelo adaptador.
4. As réplicas que têm o usuário conectado entregam a mensagem.

Como o evento passa pela fila, uma falha no WebSocket não derruba a ação que o originou.

3.5 Falhas e limites

- **Conexão caída:** o Socket.IO reconecta sozinho. Ao reconectar, o cliente busca o estado atual pela API REST, para não perder eventos ocorridos durante a queda.
- **Balanceador:** o ALB usa sessão fixa (sticky session) para o modo de contingência por long polling do Socket.IO.
- **Capacidade:** o número de conexões simultâneas por réplica será medido no teste de carga com k6, junto com os testes da Estratégia de Escalabilidade.
- **Sem WebSocket:** o web continua funcionando por atualização das consultas (React Query). O tempo real é um ganho, não um requisito para o sistema operar.

4. Decisões em aberto

> As decisões abaixo serão tomadas pela equipe nas próximas etapas. O desenho das seções anteriores funciona com qualquer uma das opções.

Decisão	Opções	Impacto
De quem é a conta do Mercado Pago	Cada condomínio conecta a própria conta, ou o Zeluvi recebe e repassa	Fluxo do dinheiro e responsabilidade jurídica
Token do WebSocket vencido	Cliente renova o token e reautentica a conexão, ou o servidor encerra a conexão	Comportamento de conexões longas (o JWT dura 15 min)
Cobrança mensal	Criada pelo síndico ou por rotina automática	Volume de eventos no início do mês
Regra de reservas	Confirmação automática (Fase 2, RN06) ou aprovação do síndico (System Design)	Evento `reserva.atualizada`
Detalhes da API do Mercado Pago	Confirmar na documentação oficial o texto exato assinado no HMAC e o nome do header de idempotência na criação do pagamento	Implementação da validação e das tentativas seguras